GraphGate: Toward an In-Loop Security Gate for Iterative LLM Code Refinement Using Temporal Code-Graph Deltas

Janani R
Department of Artificial Intelligence and Data Science
Karunya Institute of Technology and Sciences
Coimbatore, India
email address

Rajkumar
Project Guide, Department of Artificial Intelligence and Data Science
Karunya Institute of Technology and Sciences
Coimbatore, India
email address

***Abstract*—Developers rarely accept the first answer a code assistant gives them; they ask a large language model (LLM) for one revision after another. A recent controlled study found that critical vulnerabilities grew by 37.6% after only five such rounds [1]. Checks that inspect only the changed lines miss many of these regressions, because the vulnerable data flow often starts or ends in a file the revision never touched. We present GraphGate, a security gate that sits inside the refinement loop, compares the code graphs of consecutive revisions and applies four deterministic rules to their difference, using an LLM only to triage and explain what the rules flag. This paper reports the design, a refinement harness whose traces replay byte for byte, an experimental setup that runs on an open-weight code model, a benchmark built from real injection-class fixes, and a first run of the gate on it. From 775 Python security advisories we selected 12 permissively licensed projects, verified every fix link with two independent reviewers, and extracted 34 candidate regression events, labelled local or cross-file by a pre-registered rule, of which 30 were validated at author sign-off. Taken as single changes, 1 of these 30 regressions gives pattern scanners a new finding, and the graph rules flag 11. We then recorded 90 refinement runs of an open-weight model over the events and replayed no gate, two diff-only gates and GraphGate on the same recordings. Of the 63 cross-file regressions placed in those runs, the diff-only gate blocked 3 on the turn they arrived and GraphGate 14, while GraphGate blocked 12.8% of clean turns and added a median of 7.9 seconds per turn, both within limits fixed in advance. Over the cross-file events GraphGate's per-event recall is higher than the diff-only gate's in 7 and lower in none (two-sided Wilcoxon $p = 0.016$), though most regressions still pass. A taint-analysis baseline, the ablations and the holdouts remain.**

***Index Terms*—large language models, secure code generation, iterative refinement, taint analysis, code property graphs**

I. Introduction

AI coding assistants are used conversationally. A developer asks for a function, reads it, and then asks again: make it faster, add a feature, tidy it up. Most of what we know about the security of generated code, however, comes from judging single responses [2], [3]. Shukla et al. [1] studied the loop itself: across 400 code samples and 40 rounds of requested improvements, security got worse as the rounds went on. They deliberately left out any intervention and named verification inside the loop as the follow-up that was needed.

The obvious intervention is to check each revision before accepting it, but practical checks are local to the change: a pattern scanner such as Semgrep [4] or Bandit [5] run on the diff, or an LLM reviewer shown the changed lines. Many injection flaws are not local. A revision can weaken a validation helper that other modules rely on, change a default so that a distant call becomes unsafe, or open a call path that routes user input around an existing guard. The diff can then look harmless, because the danger lies in code the diff does not show.

GraphGate represents each revision as a code graph of symbols, calls, taint edges and sanitizers, and reasons about the *difference* between the graphs of consecutive revisions. Four deterministic rules look for sanitizer removal, new source-to-sink paths, eliminated guards and shortened paths, and an LLM then classifies each flag and writes a rationale that can be fed into the next prompt. Keeping detection in deterministic rules makes it testable, and keeping the LLM in a triage role limits the room it has to invent findings.

Our research question is whether this cross-file grounding reduces security degradation compared with no gate and with a gate restricted to the diff. Answering it needs a harness whose runs can be replayed exactly, a benchmark of real regressions with trustworthy labels, and a protocol fixed in advance. This paper contributes:

- a gate design based on temporal code-graph differencing, with a deliberately narrow role for the LLM, and an evaluation protocol fixed in advance (Section III);
- a benchmark of real injection regressions with independently verified fix links and a pre-registered local/cross-file rule (Section IV);
- a refinement harness with byte-identical replay, the models used and the evidence for choosing them, and first measurements on the benchmark (Section V).

The graph layer and its four rules are implemented and unit-tested, and the gate conditions have been run once on the recorded traces. The taint-analysis baseline, the ablations and the holdouts have not been run; for these, every expected outcome is marked as an expectation.

II. Literature Survey

Security of generated code. Pearce et al. [2] found that roughly 40% of programs GitHub Copilot produced for scenarios built on MITRE's dangerous weaknesses [6] were vulnerable. In a user study by Perry et al. [3], participants with an AI assistant wrote less secure code while feeling more confident about it. These studies evaluate single responses. Iteration is usually presented as a way to improve output, although the gains of self-repair for code are modest and bounded by the quality of the feedback [7]; Shukla et al. [1] showed that it can also erode security, with distinct vulnerability patterns for different prompting strategies.

Static analysis and code graphs. Taint analysis reports flows of untrusted data that reach a sensitive sink without passing a sanitizer. Yamaguchi et al. [8] merged syntax trees, control flow and program dependences into a queryable code property graph. For Python, Pysa [9] performs taint analysis on top of the Pyre type checker, and CodeQL [10] offers data-flow queries. These tools analyse snapshots; they can be rerun after every revision, and we use them as baselines, but they do not reason about what changed or explain findings in a form that can steer the next step.

Learning-based and LLM-based detection. Early detectors trained graph neural networks or transformers on labelled functions, but much of their reported accuracy does not survive realistic evaluation, partly because datasets contain label noise and duplicates [11]. Recent systems pair LLMs with program analysis: IRIS [12] infers taint specifications for a static analyser, LLMxCPG [13] gives an LLM compact graph slices and cuts the code it must read by 67.84–90.93%, and RepoAudit [14] explores whole repositories with an agent. All of them look at code at one point in time.

Benchmarks and reproducibility. CVEfixes [15] pairs vulnerabilities with fixing commits, but automatically collected links can be wrong [11]; we take advisories from OSV [16], which aggregates the GitHub Advisory Database, and verify every link. LLM code generation is non-deterministic even at temperature zero [17], and hosted models change over time; our design therefore relies on replayed traces. Table I summarises the gap: we are not aware of prior work that evaluates a graph-grounded gate inside an LLM refinement loop.

TABLE I
Position of GraphGate relative to prior work

Work	Setting	Context used	Across revisions
Shukla et al. [1]	Iterative refinement	None (measurement)	Measures only
Pysa, CodeQL [9], [10]	Static taint analysis	Whole program	No
LLMxCPG [13]	LLM with graph slices	CPG slice	No
IRIS, RepoAudit [12], [14]	LLM with analysis or agent	Repository	No
GraphGate (ours)	In-loop gate	Graph delta around changes	Yes

III. Methodology

Research questions. The project is organised around four questions. **RQ1:** does a graph-differencing gate reduce security degradation across refinement iterations, compared with no gate and with an otherwise identical diff-only gate? **RQ2:** what is its recall on cross-file regressions, and how many iterations pass before one is caught? **RQ3:** does it outperform mature static taint analysers used alone? **RQ4:** what does it cost in false alarms, latency and tokens, and which graph components matter?

Gate overview and deployment. Fig. 1 shows the gate. Before a candidate revision is accepted, it runs Semgrep and Bandit on the diff, then the graph-difference engine, then LLM triage of whatever was flagged. A blocked revision is not committed and its rationale joins the next prompt; an allowed revision is committed and its files are re-indexed into the graph. The targets are a pre-merge check in continuous integration and middleware in an agent framework, for example a LangGraph node between the generation and commit steps. Both tolerate a few seconds of added latency; interactive editor use, with its tighter budget, is out of scope.

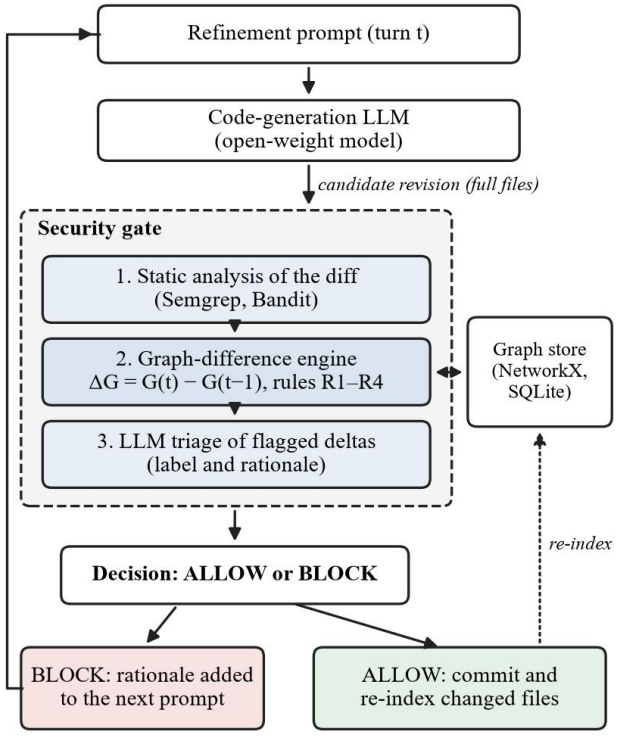


Fig. 1. GraphGate inside the refinement loop. The LLM only triages what the deterministic stages flag.

Code graph. The graph of revision t is $G(t) = (V, E_{\text{call}}, E_{\text{taint}}, E_{\text{sanitize}})$. Vertices are function, method and module symbols from tree-sitter [18] syntax trees. Call edges are resolved only for calls within a module, calls through static imports and attribute calls whose receiver class is statically known; any other call becomes an explicit *unknown* edge, so the analysis can report what it could not see. Taint edges link sources

(request parameters, environment variables, file and network input, command-line arguments) to uses through assignments and call arguments, flow-insensitively across procedures. Sanitizers are symbols matching an allowlist seeded from Semgrep and Pysa rules. Graphs are held in NetworkX and persisted between iterations.

Difference rules and triage. After each revision the engine computes $\Delta G = G(t) - G(t-1)$ within two hops of the changed symbols; the depth is an ablation variable. Four rules are applied to the difference.

- **R1, sanitizer removal:** a sanitizer present before is absent or bypassed while its callers remain.
- **R2, new path:** a source-to-sink path exists that did not exist before.
- **R3, guard elimination:** a guard on an existing source-to-sink path disappears.
- **R4, path shortening:** the shortest sanitizer-free path from a source to a sink gets shorter.

Each rule is a pure function of two graphs and is unit-tested on hand-built graphs before it runs on real code. The triage LLM receives each flag with its supporting subgraph and labels it an exploitable regression, a benign refactor or uncertain. It is never asked to search the code on its own, which keeps the detector testable and narrows the room for invented findings.

Scope. The scope is Python and the injection family: SQL injection (CWE-89), OS command injection (CWE-78), command injection in general (CWE-77) and path traversal (CWE-22) [6]. The graph does not model monkey-patching, reflection or framework-generated routing, so results will describe practical, lightweight differencing rather than a sound analysis, and unknown-edge counts will be reported per repository.

Conditions and baselines. The gate is evaluated on recorded refinement traces (Section IV). All conditions in Table II consume the same traces, so only the gate's context and detector vary. B+ was added once the first measurement (Section V) showed how rarely the scanners fire: there the LLM also judges every change from its diff, so that C faces a strong diff-only check as well. It is a baseline only; in C the LLM still triages flags and nothing else.

TABLE II
GATE CONDITIONS AND BASELINES

Condition	Configuration	Question
A	No gate	Baseline degradation
B	Semgrep, Bandit and LLM triage on the diff	Conventional check
B+	B, with the LLM judging every diff as a whole	Strong diff-only check
C, GraphGate	B plus the graph-difference engine	RQ1, RQ2
S1, S2	Pysa; CodeQL if licensing allows (no LLM)	RQ3

Metrics, ablations and success criteria. We will report the degradation curve, cross-file recall, iterations to detection, the false-positive burden (BLOCK decisions on clean turns), latency, tokens, cost and the refusal rate. Traces are replayed over several replications, and C is compared with B, B+ and S1 by paired Wilcoxon signed-rank tests [19] on per-trace recall, with effect sizes. Ablations remove call or taint edges, vary the hop depth from one to three, drop LLM triage and swap in the smaller model.

Scoring a replay. Traces are recorded with no gate, so each turn is judged as recorded and a BLOCK changes nothing downstream. A regression counts as caught from the first BLOCK on a turn where it is present, and a BLOCK on a clean turn is a false block. What the model would have written after a block is not simulated. Condition C is registered with its graph built from the slice and the rest of the repository; C on the slice alone is reported beside it.

The success criteria were fixed in advance: condition A reproduces the degradation trend of the base study, condition C improves cross-file recall over B significantly and is competitive with S1, and C keeps false positives below about 15% of clean turns with a median added latency under about 10 seconds. A null result on the cross-file comparison would still be reported, because the controlled design is part of the contribution.

IV. DATASET DESCRIPTION

Each event is a real fix, inverted: the fixed code is the clean state and the vulnerable code is the regression that will be injected into a refinement trace. The targets are 25 to 30 validated events for the first milestone and 40 to 60, a third or more of them cross-file, for the full study. Fig. 2 traces the pipeline.

Advisories and seed repositories. From the OSV export for PyPI [16], retrieved on 27 September 2026 and pinned by its hash, we kept reviewed GitHub advisories (only these carry CWE identifiers) with an injection CWE, giving 775 advisories, 468 of them linking a fix, across 243 repositories. Seed repositories must be MIT, Apache-2.0 or BSD licensed, so that traces can be redistributed, and hold 5,000 to 50,000 lines of Python outside tests and documentation. We selected 12 (Table III) and kept 4 in reserve. Path traversal dominates, so results will be reported per class.

TABLE III
SELECTED SEED REPOSITORIES

Repository	Licence	Advisories
gitpython-developers/gitpython	BSD-3-Clause	12
parisneo/lollms_legacy	Apache-2.0	9
zauberzeug/nicegui	MIT	4
datadog/guarddog	Apache-2.0	3 (excluded: size)
ietf-tools/xml2rfc	BSD-3-Clause	3
mar10/wsgidav	MIT	2
sqlalchemy/mako	MIT	2
alerta/alerta	Apache-2.0	1
collerek/ormar	MIT	1
geopython/pycsw	MIT	1
piccolo-orm/piccolo	MIT	1
tortoise/tortoise-orm	Apache-2.0	1

Fix pairs and their verification. The resolver takes the earliest linked commit on the default branch as the fix and its first parent as the vulnerable version, and re-measures size and licence at that commit; this removed one repository whose code was only 1,200 to 4,200 lines at every vulnerable commit. Two independent reviewers then checked each pair, one asked to establish the link and one to refute it, and advisories without a linked fix got a history search. Of 40 items, 30 were confirmed outright. Reading every other case against the history, we corrected 4 pairs where both reviewers named the same other commit and recovered 4 missing fixes. The simple linking rule was thus wrong for about one advisory in eight. Corrections and recoveries are kept in separate files so that every commit not taken from an advisory can be audited.

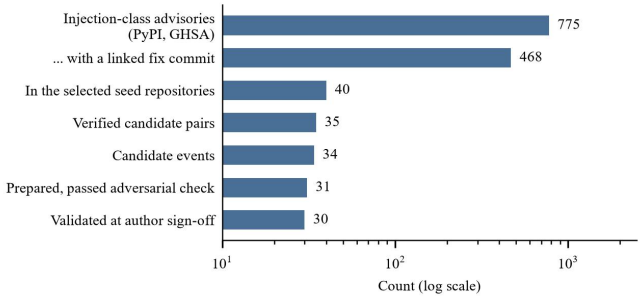


Fig. 2. Dataset funnel, from advisories to validated events.

Event extraction. The model rewrites whole files, and 16 of the 35 candidate pairs have a file on the vulnerable flow of more than about 14,000 tokens. Each event is therefore a *trimmed slice*: the files on the flow at their real paths, cut down to the functions between the entry point, the guard the fix added and the sink. A clean version is cut at the fix commit and a regressed version at the vulnerable commit. The slicer keeps source text verbatim and reports every referenced name it removed, and comments that describe the vulnerability or the fix are stripped from both versions and logged, because the model sees every file verbatim.

Scope label and validation. An event is *local* if entry, guard and sink lie in one file, and *cross-file* otherwise; this rule was fixed before any event was built, and the label follows the real sink even when a large sink function is represented by its call site. For each event an LLM analyst builds the slice, an independent LLM checker tries to refute it, and only the authors' sign-off validates it. 31 events passed the check and the authors accepted 30 of them, 24 cross-file (Fig. 3); the sign-off reviewed the prepared dossiers and did not re-derive them. Of the 4 events not accepted, one is a finding in itself: our analysis indicates that a published fix sanitises a document that the command-line PDF path then discards and re-parses, leaving that path vulnerable. Another reflected a limit of our own tooling: the slicer could not keep a module-level block that connects an upload handler to its sink, and has since been extended to keep such wiring on request.

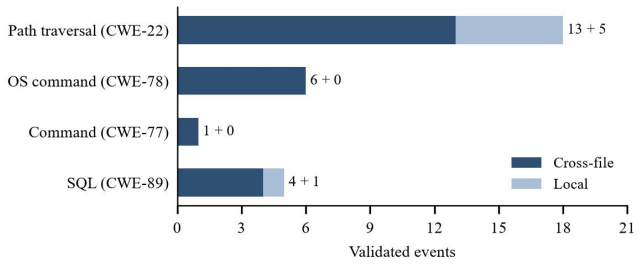


Fig. 3. Validated events by vulnerability class and scope.

Traces, labels and leakage control. Each validated event becomes a refinement trace of five to eight turns (improve readability, add logging, optimise). At a turn between the second and the sixth, fixed by a seeded plan, the regression is merged into the code the model has just produced, and every turn is labelled clean, local regression or cross-file regression from the recorded code. Near-duplicates of the regressed code elsewhere in the repository and tests changed by the fix are kept out of the graph's retrieval scope to avoid leakage; the scan found exact twins in one event and 32 such test files. All 30 events were recorded with the open-weight model of Section V, three runs each: 584 turns in 90 runs.

V. Implementation and Results

Refinement harness. The harness drives a refinement loop over a set of Python files. Each turn sends the model a fixed system prompt, every current file and one instruction; the model returns complete files, and the harness computes a diff and appends a record to a JSON Lines trace holding the prompt and its hash, the requested and the served model identifiers, the parameters, the response, token usage, latency and the diff. Refusals and errors have their own record kinds. Each prompt sequence is run several times, and every run carries a replication number. The code is Python 3.12 with pinned dependencies and 643 automated tests.

Reproducibility without sampling controls. Sampling settings do not make a model reproducible: hosted models increasingly reject them, and neither a fixed seed nor a zero temperature guarantees identical output [17]. Reproducibility therefore rests on a response cache keyed by (prompt hash, replication) and on replay: the gate conditions never call the model but consume recorded traces, so every condition sees the same revisions. Recorded traces replay byte for byte, those of the open-weight model included.

Two pitfalls are worth reporting because they fail silently. Our first cache used the prompt hash alone as its key; replications send identical requests, so the cache served the first replication's answer to the others and N replications collapsed into one. Adding the replication number to the key fixed this, and a regression test guards it. Second, we passed the replication number to the model server as its sampling seed. Under one seed the model's reply then broke the output format in all 14 events a session started, while two other seeds never did; we attribute this to a fixed seed reusing one random stream for every request. That seed was replaced, on format validity alone, before any gate was run.

Model used. The experiments run on an open-weight model, so that they do not depend on a commercial service and anyone can repeat them: Qwen2.5-Coder with 14.8 billion parameters [20], released under Apache-2.0, in 4-bit quantised form with a 32,768-token context, served by Ollama on a free cloud GPU. The same model performs the triage step. Over the dataset it returned complete, usable files in all 584 turns, at a median of 161 seconds per turn. A turn the model declines or answers unusably is stored as an outcome, ends its run and never triggers a fallback to another model, which would mix models within a condition; no such turn occurred in the recorded runs. Table IV collects the other measurements.

TABLE IV
RESULTS OBTAINED SO FAR

Measurement	Result
Automated tests	643, all passing
Replay of recorded traces	Byte-identical, all 30
Events accepted at sign-off	30, of which 24 cross-file
Runs with the regression placed	80 of 90
Turn labels carried over	302 of 584
Kept out of retrieval scope	3 code twins, 32 test files

Static analysis of a change. The first stage of the gate is implemented. Semgrep 1.179.0 and Bandit 1.9.4 scan the files a turn changes, before and after it, and a finding counts only if the change introduces it. Semgrep uses its public community rules, pinned at one commit and restricted to the injection family, 63 rule files in all. As a first measurement we took each validated event's regression as a single change. Only 1 of the 30 regressions introduces a finding, a local SQL event; for the others the scanners report the same lines before and after, or nothing. Path traversal accounts for 18 of the events but only 5 of the rule files, all tied to particular web frameworks. No model turn or triage is involved here; it shows how little of a removed guard a pattern scanner sees in the changed lines. One triage prompt, fixed before any live call, serves every condition.

Graph layer and a first look. The graph builder and rules R1 to R4 are implemented, with 73 unit tests. On the 12 seed repositories the builder resolved all but 35% of 69,722 calls, which stay in the graph as unknown edges. Taint edges join the parameters and return values of symbols, not whole functions. Two settings depart from our proposal and were fixed before the rules saw any event: the parameters of a library's public functions count as sources, and a test that raises counts as a guard. Taking each regression as one change, with the rest of its repository laid around the slice, the rules flag 11 of 30, 8 of the 24 cross-file ones, against 1 for the scanners; on the slice alone they flag 8, and with the proposal's literal settings 7. As a control, the rules also fire on 5 of the reverse changes, the fixes. The events the repository adds are regressions whose sink the slice had cut away. Most misses are checks weakened inside a function, which remove no sanitizer and open no path, or sinks reached through dynamic dispatch. This first look involves no model turn and no triage, and neither the rules nor their settings were changed after it.

What the recording showed. A smaller open model reshapes code freely. It rewrote the functions a regression lives in so often that 302 of 584 turn labels had to be carried over from the previous turn instead of being read off the code, and the regression could not be placed in 10 of 90 runs because its function had been removed or renamed, in one event in all three. Both are reported and neither is worked around.

TABLE V
GATE CONDITIONS ON THE RECORDED RUNS: REGRESSIONS BLOCKED ON THE TURN THEY ARRIVE, CLEAN TURNS BLOCKED, AND MEDIAN ADDED TIME

Condition	All (of 80)	Cross-file (of 63)	Clean turns (of 235)	Time (s)
A, no gate	0	0	0	–
B	5	3	8 (3.4%)	6.5
B+	7	3	7 (3.0%)	11.7
C, slice only	16	9	21 (8.9%)	6.6
C	22	14	30 (12.8%)	7.9

Gate conditions on the recorded runs. All five conditions were run once over the 90 runs, with the model's triage (Table V). The regression was placed in 80 runs, 63 of them cross-file. On the turn it arrives, B blocks 5 and GraphGate 22; of the cross-file ones B and B+ each block 3, GraphGate on the slice 9 and GraphGate with the repository 14. Counting later turns too, B blocks 13 of the 80 before the run ends and GraphGate 33, so 47 outlast the gate. By class GraphGate blocks 9 of 14 SQL regressions on arrival, but 10 of 47 path-traversal and 3 of 19 command ones. The price is false blocks: 30 of 235 clean turns (12.8%) against 8 for B. Its median added time, measured on 60 turns, is 7.9 seconds, 6.4 of them the scanners; a turn whose flags go to triage waits a median of 9.0 seconds more. Both registered limits, about 15% of clean turns blocked and about ten seconds, are met, the first narrowly. Recall is given on the arriving turn because that label is always read off the code.

Significance. The test was planned before it was computed: per-event recall at introduction, a two-sided exact Wilcoxon signed-rank test, events without a difference dropped and counted, level 0.05. Of the 23 cross-file events with a placed regression, GraphGate's recall is higher than B's in 7 and lower in 0; 16 are tied, 15 of them because both gates miss every run. The registered comparison gives $p = 0.016$, a mean per-event recall of 0.22 against 0.04 and a difference of 0.17 (95% bootstrap interval 0.07 to 0.30). It holds over all events and against B+ (Holm-adjusted p at most 0.016). The result rests on the 7 events where the gates differ: it shows that the graph catches regressions the diff-only gates do not, not how large the gain is in general.

Expectations against measurements. B's cross-file recall was expected near zero, and it is 3 of 63. B+ was expected to catch more than B at the price of false blocks; it catches no more cross-file regressions and blocks as few clean turns, so reading the whole diff added little. GraphGate was expected to exceed B on cross-file recall by less than hoped: it does, significantly, yet lets most regressions through. Added time was expected to pass ten seconds and did not for B and C, since most turns raise no flag and need no model call; B+, which calls the model on every turn, takes 11.7 seconds. In condition A every placed regression lasts to the end of its run

as far as the labels show; that curve is the reference for the gates, not a replication of the base study's trend. *Threats to validity. Construct:* a regression is defined as an inverted real fix, and only the injection family is covered; the local/cross-file label follows one operational rule, which is why it was fixed before the events were built. *Internal:* LLM non-determinism is controlled by replay. LLM agents assisted curation, so every such step has an adversarial check and author sign-off; stripped comments may still leave cues such as identifiers that mention unsafe options. A label carried over from an earlier turn is not verified ground truth, and such turns are counted separately. *External:* injected regressions in synthetic traces may differ from natural ones, so a holdout of real multi-turn sessions, reviewed manually and never used for tuning, is planned. The dataset is dominated by path traversal and one library supplies two fifths of the events. *Conclusion:* the number of events is modest; paired non-parametric tests, effect sizes and oversampling of cross-file events keep conclusions conservative. Public advisories may also appear in model training data.

TABLE VI
REMAINING STEPS, WHAT EACH MEASURES AND THE EXPECTED OUTCOME

Step	**Measures**	**Expected (not yet measured)**
Pysa baseline S1	Taint flows per revision; paired test against C	Silent on what changed
Ablations	Edge types, hop depth, no triage, literal settings	Literal settings catch less
Cross-file variants	15 to 25 added events	Recall close to that on the present events
Real-session and repository holdouts	Recall on data never used for tuning	Smaller gain than on synthetic traces
Error analysis	Misses and false blocks by cause	Weakened checks, unknown edges

VI. CONCLUSION AND FUTURE WORK

GraphGate reasons over the difference between code graphs of successive revisions and limits the LLM to triage. We have presented its design, a harness with exact replay, an open-weight experimental setup, a verified benchmark of 30 regression events with a pre-registered scope rule, and a first run of the gate. On 90 recorded runs GraphGate blocked 14 of 63 cross-file regressions on arrival where the diff-only gate blocked 3, within the registered limits on false blocks and added time. The difference is significant ($p = 0.016$) but rests on the 7 events where the gates differ, and most regressions still pass.

Table VI is the plan for future work: the Pysa baseline completes the comparison reported here, and the ablations, added cross-file events and the two holdouts follow. Traces, labels, rules and the harness will be released under a permissive licence once the licence check of every seed repository is complete.

The authors thank the Karunya Institute of Technology and Sciences for its support. In line with IEEE policy, we disclose that Anthropic's Claude, through Claude Code, assisted with implementing the software, with the dataset verification in Section IV, and with drafting this manuscript; the authors reviewed all of it and take full responsibility for the content.

REFERENCES

[1] S. Shukla, H. Joshi, and R. Syed, "Security Degradation in Iterative AI Code Generation: A Systematic Analysis of the Paradox," in *Proc. IEEE International Symposium on Technology and Society (ISTAS)*, 2025. arXiv:2506.11022.

[2] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions," in *Proc. IEEE Symposium on Security and Privacy (SP)*, 2022, pp. 754–768.

[3] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do Users Write More Insecure Code with AI Assistants?" in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023, pp. 2785–2799.

[4] Semgrep, Inc., "Semgrep." Accessed: Sep. 2026. [Online]. Available: https://semgrep.dev

[5] PyCQA, "Bandit: A Tool Designed to Find Common Security Issues in Python Code." Accessed: Sep. 2026. [Online]. Available: https://github.com/PyCQA/bandit

[6] The MITRE Corporation, "Common Weakness Enumeration (CWE)." Accessed: Sep. 2026. [Online]. Available: https://cwe.mitre.org

[7] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama, "Is Self-Repair a Silver Bullet for Code Generation?" in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[8] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and Discovering Vulnerabilities with Code Property Graphs," in *Proc. IEEE Symposium on Security and Privacy (SP)*, 2014, pp. 590–604.

[9] Meta Platforms, "Pysa: Static Analysis for Security in Python." Accessed: Sep. 2026. [Online]. Available: https://pyre-check.org/docs/pysa-basics/

[10] GitHub, "CodeQL." Accessed: Sep. 2026. [Online]. Available: https://codeql.github.com

[11] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, *et al.*, "Vulnerability Detection with Code Language Models: How Far Are We?" in *Proc. IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025.

[12] Z. Li, S. Dutta, and M. Naik, "IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities," in *Proc. International Conference on Learning Representations (ICLR)*, 2025.

[13] A. Lekssays, H. Mouhcine, K. Tran, T. Yu, and I. Khalil, "LLMxCPG: Context-Aware Vulnerability Detection Through Code Property Graph-Guided Large Language Models," in *Proc. 34th USENIX Security Symposium*, 2025, pp. 489–507.

[14] J. Guo, C. Wang, X. Xu, Z. Su, and X. Zhang, "RepoAudit: An Autonomous LLM-Agent for Repository-Level Code Auditing," in *Proc. 42nd International Conference on Machine Learning (ICML)*, 2025.

[15] G. Bhandari, A. Naseer, and L. Moonen, "CVEfixes: Automated Collection of Vulnerabilities and Their Fixes from Open-Source Software," in *Proc. 17th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE)*, 2021, pp. 30–39.

[16] Open Source Security Foundation, "OSV: A Distributed Vulnerability Database for Open Source." PyPI export retrieved 27 Sep. 2026. [Online]. Available: https://osv.dev

[17] S. Ouyang, J. M. Zhang, M. Harman, and M. Wang, "An Empirical Study of the Non-Determinism of ChatGPT in Code Generation," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 2, 2025. Art. 42, doi: 10.1145/3697010.

[18] Tree-sitter contributors, "Tree-sitter: A Parser Generator Tool and Incremental Parsing Library." Accessed: Sep. 2026. [Online]. Available: https://tree-sitter.github.io

[19] F. Wilcoxon, "Individual Comparisons by Ranking Methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[20] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, *et al.*, "Qwen2.5-Coder Technical Report." arXiv:2409.12186, 2024. [Online]. Available: https://arxiv.org/abs/2409.12186